

Seabed Substrate Classification

Jonas Schwarzhaupt, Anne Maring, Babak Behkamkia

July 2026

1 Introduction

The aim of this project was to classify images into the three categories bare soil, stones, and vegetation. Since only a limited number of labeled images was available, the main challenge was to get good classification results although only a small dataset was available.

To investigate different solutions, two main approaches were evaluated. First, manually extracted image features were used as input for a classification model. In parallel, deep learning methods were tested, including a convolutional neural network trained from scratch and a transfer learning approach. Image augmentation was applied to increase the amount and variability of the available training data.

2 Methodology

2.1 Data Augmentation

Due to the limited amount of available data, image augmentation was applied to increase the size and variability of the dataset. Before augmentation, fully black borders were removed from all images.

For the deep learning approach, six augmented versions were created from each original image: one horizontally flipped image, one image rotated by 15 degrees, one image with increased brightness, one image with increased contrast, one blurred image, and one cropped and subsequently resized image. Together with the original image, this resulted in seven images per input image.

For the feature extraction approach, the preprocessing steps neutralized the brightness and contrast transformations, which resulted in multiple nearly identical feature representations. Therefore, an adapted augmentation was created and then used. The brightness and contrast transformations were replaced by a vertical flip and a vertically flipped, cropped and resized image. This preserved a higher variation between the extracted features.

2.2 Feature Extraction

For the feature extraction approach, each image was first preprocessed before extracting the features. A Gaussian blur was applied to reduce noise, followed by CLAHE. It was applied only on the brightness channel in the LAB color space, so that the colors of the image were not affected. (Figure 1)

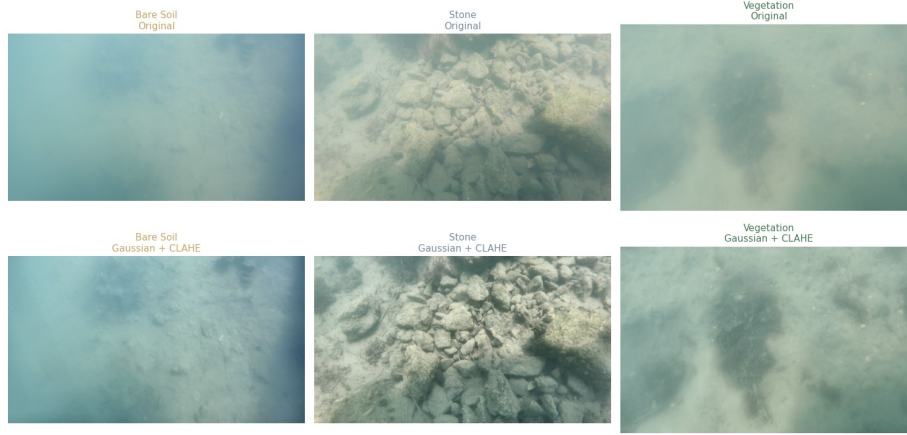


Figure 1: The effect of preprocessing on the images of each class

Two types of features were then extracted from each preprocessed image. The first type are color features, extracted from the HSV and LAB color spaces. For each of the six channels (three from HSV and three from LAB), a normalized 32-bin histogram was computed. This resulted in 192 color features per image. The second type are gradient features, which describe the edges and texture of the image. These were computed from the grayscale version of the preprocessed image using Sobel and Laplacian. (Figure 2)

In total, each image was represented by a feature vector of 195 values, which were saved to a CSV file for later use by the classification model.

2.3 Deep Learning

In total we tried three different methods using a deep learning approach for the classification of the images.

First we tried using the augmented dataset on a trained model from scratch. Then we tried a different deep learning model on the extracted features and finally we applied transfer learning to a publicly available image classification dataset, using the normal images.

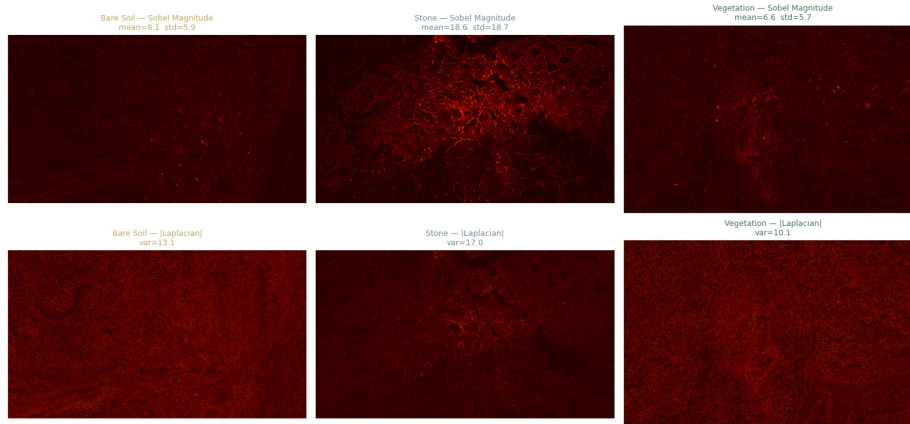


Figure 2: An example of the extracted gradient features

3 Implementation

3.1 Data Augmentation

The data augmentation was implemented in C++ using OpenCV. First, fully black borders were removed by detecting all non-black pixels and cropping the image to the smallest bounding rectangle containing them. The images were then processed separately for the three classes bare soil, stones, and vegetation. For the deep learning approach, the original image and six augmented versions were saved. These consisted of a horizontal flip, a rotation by 15 degrees, increased brightness, increased contrast, Gaussian blur, and a crop that retained 80% of the image before resizing it to the original dimensions.

For the feature extraction approach, an adapted version was implemented. The brightness and contrast transformations were replaced by a vertical flip and a vertically flipped image that was cropped and resized. All generated images were stored in separate class folders for further processing.

3.2 Deep Learning

3.2.1 Dataset Preparation

The dataset needs further preparation to be given as an input to the classification model.

First the images were resized to a uniform, quadratic shape of (224 x 224) since not all images had the same size from the beginning. For images that were smaller than the given frame, a padding was added.

Then the images needed to be transformed. For the convolutional model the images were transformed into gray-scale. This was done because there was only a limited amount of data available and we wanted to reduce the complexity of

the individual images.

For the transfer learning model the images needed to fit the requirements of the original model. Therefore a transformation from BGR to RGB was applied to the reshaped original images.

For each image in both datasets a normalization was also applied to the image. To accelerate the training, the images were given in batches of 4.

3.2.2 Convolutional Model

The convolutional model was built from scratch. The model consists of 3 convolutional layers, followed by max-pooling, to automatically reduce the complexity of the images. The number of filters doubled with each layer, starting from 32. The kernel size for each convolutional layer is (3×3) and (2×2) for max-pooling. The activation functions are ReLU, except for the last layer, where softmax is applied. For the training, the adam optimizer was used and the loss-function is the categorical cross-entropy.

The model was trained with 200 epochs and only on the training data, which consist of 80 percent of the augmented dataset. The remaining 20 percent were used for the testing of the model, where 20% were taken from each class.

3.2.3 Transfer Learning

For the Transfer Learning the MobileNetV2 was used [1].

The model is a 53-layer CNN model that takes images of the size (224×224) as an input. It was trained on the ImageNet which is a dataset containing 1.4 million images with around 1,000 categories.

The model itself was not trained again, however we adjusted the output layer though adding a global average pooling and a final dense layer with softmax for the classification of three outputs.

For the training 200 epochs were used and no validation data, due to too few data in total.

The train-test split was done the same way as in 3.2.2.

3.2.4 Feature-Based Neural Network

A simple FFNN is used for classification using the extracted features from each image. We tried to keep the model as simple as possible in order to prevent possible overfitting. The implemented model consists of only two Dense layers with Adam optimizer and SparseCategoricalCE loss function. Before training, the features were normalized using a StandardScaler. This was done to ensure that all 197 features are on the same scale, which helps the network train more effectively. The data was split into 80% training and 20% testing.

Metrics	CNN	Feature Model	MobileNetV2
Accuracy	0.9661	0.881	0.9491
F1-score	0.9646	0.852	0.9469

Table 1: Results of the classification models

4 Results

The results were evaluated with the accuracy and the F1-score. The CNN that was trained from scratch results into a better accuracy score with 96.61 % than the MobileNetV2 with 94.91 %. The F1-score however shows only a difference of 0.0177, which is negligible.

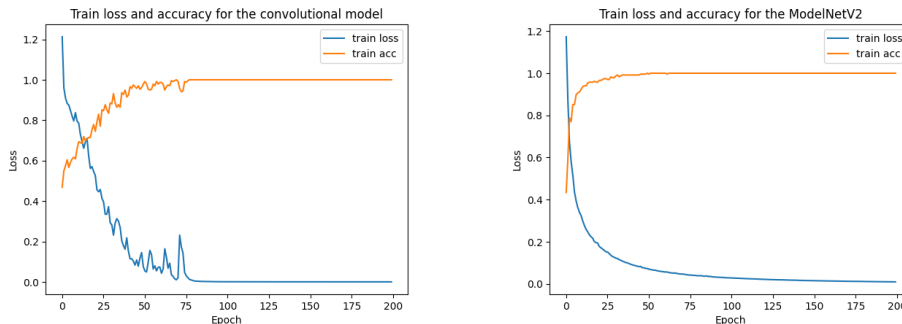


Figure 3: Comparing training of CNN and Transfer-Learning

Both models reach an accuracy of 100% on the training data and a loss close to zero. While comparing the training trajectories there are visible fluctuations for the loss and accuracy. This could be due to the mini batching.

The results of the confusion matrix indicate that the classes are well separated. Only a few samples were misclassified, but there is no evidence of systematic confusion between different image classes.

Although the proposed two models have higher results than the Feature-based Model, but it has a significantly simpler structure (consist of two dense layers). This simplicity results in lower computational complexity and faster training which reduce the risk of overfitting, particularly when working with a low amount of training data.

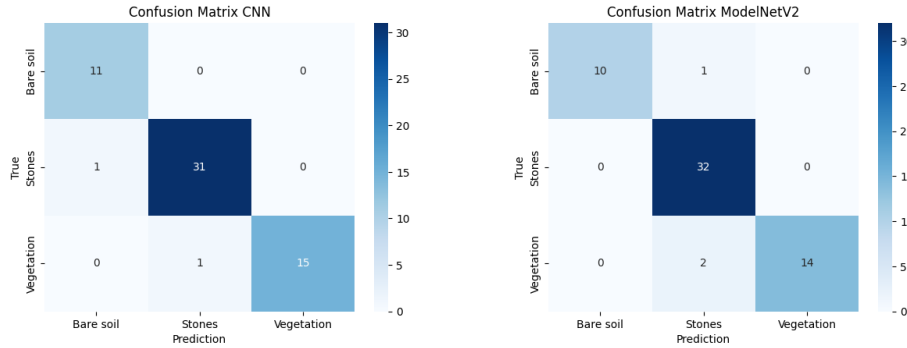


Figure 4: Confusion matrices of the CNN and MobileNetV2 models

5 Discussion

All three models achieved high classification results on the augmented dataset. The CNN trained from scratch achieved the highest accuracy and F1-score, while MobileNetV2 produced similar results using pre-trained features. The feature-based model achieved lower results, but used a considerably simpler architecture.

The training accuracy of the CNN and MobileNetV2 reached 100%, while the training loss approached zero. This indicates possible overfitting caused by the limited amount of available data. Although data augmentation increased the size and variability of the dataset, the number of original images was still too small for a reliable evaluation. Differences between the classes may also be related to the unequal amount of available ground-truth images. More original training and testing images would therefore be required to evaluate the models more reliably.

6 Team Member Contribution

Name	Hours	Responsibilities
Jonas Schwarzhaupt	12	Data Augmentation
Anne Maring	14	Deep Learning Models
Babak Behkamkia	12	Feature Extraction

References

- [1] Mark Sandler et al. “Inverted Residuals and Linear Bottlenecks: Mobile Networks for Classification, Detection and Segmentation”. In: *CoRR* abs/1801.04381 (2018). arXiv: 1801.04381. URL: <http://arxiv.org/abs/1801.04381>.